NORTHERN UNIVERSITY
OF BUSINESS & TECHNOLOGY KHULNA
Department of Computer Science and Engineering

SMART FARMER ASSISTANT

AI-Based Agriculture Support System

Submitted To:

Md. Riaz Mahmud

Assistant Professor, Department of Computer Science and Engineering

Northern University of Business and Technology Khulna

Submitted By:

Team Name: CSE4204-8D-T03

Team Leader: Sakib Foysal Ejarder

ID: 11220320948

Section: 8D

Submission date: 29th June 2026

CSE4204 — Mobile Computing Lab
Week 06 Submission

Backend Progress Report

Smart Farmer Assistant

AI-Based Agriculture Support System

Table of Contents

1. Project Title: Smart Farmer Assistant

2. Team Information

Field	Details
Team Name	CSE4204-8D-T03
Section	8D
Course	CSE4204 — Software Engineering Lab
Semester	Spring 2026
Institution	Leading University / University (Bangladesh)
GitHub Repo	github.com/sakib-foysal/CSE4204-8D-T03-SmartFarmer-Assistant

1.1 Team Members

#	Name	Student ID	Role
1	Sakib Foysal Ejarder	11220320948	Team Leader / Backend
2	Md. Noyon Sheikh	11220320946	Frontend Developer
3	Sayed Tauhidul Islam	11220320950	AI / ML Engineer
4	Sayed Akib Osman	11220320974	Database / QA

3. Backend Technology Stack

Layer	Technology
Backend Framework	Django 5.2.14
API Framework	Django REST Framework 3.17.1
Programming Language	Python
Database	PostgreSQL
Authentication	Custom JWT-style Bearer Token
Password Security	Django Password Hashing
Environment Management	python-dotenv
AI/ML Libraries	TensorFlow, OpenCV, Keras, NumPy, Pillow
API Testing	Postman
Version Control	Git and GitHub
Development Status	In Progress

4. Database Design Summary

The backend uses Django ORM models with PostgreSQL database configured through environment variables. The database design is based on the approved ER diagram created in Week 04.

4.1 ER Diagram

The complete database design showing all tables and relationships:

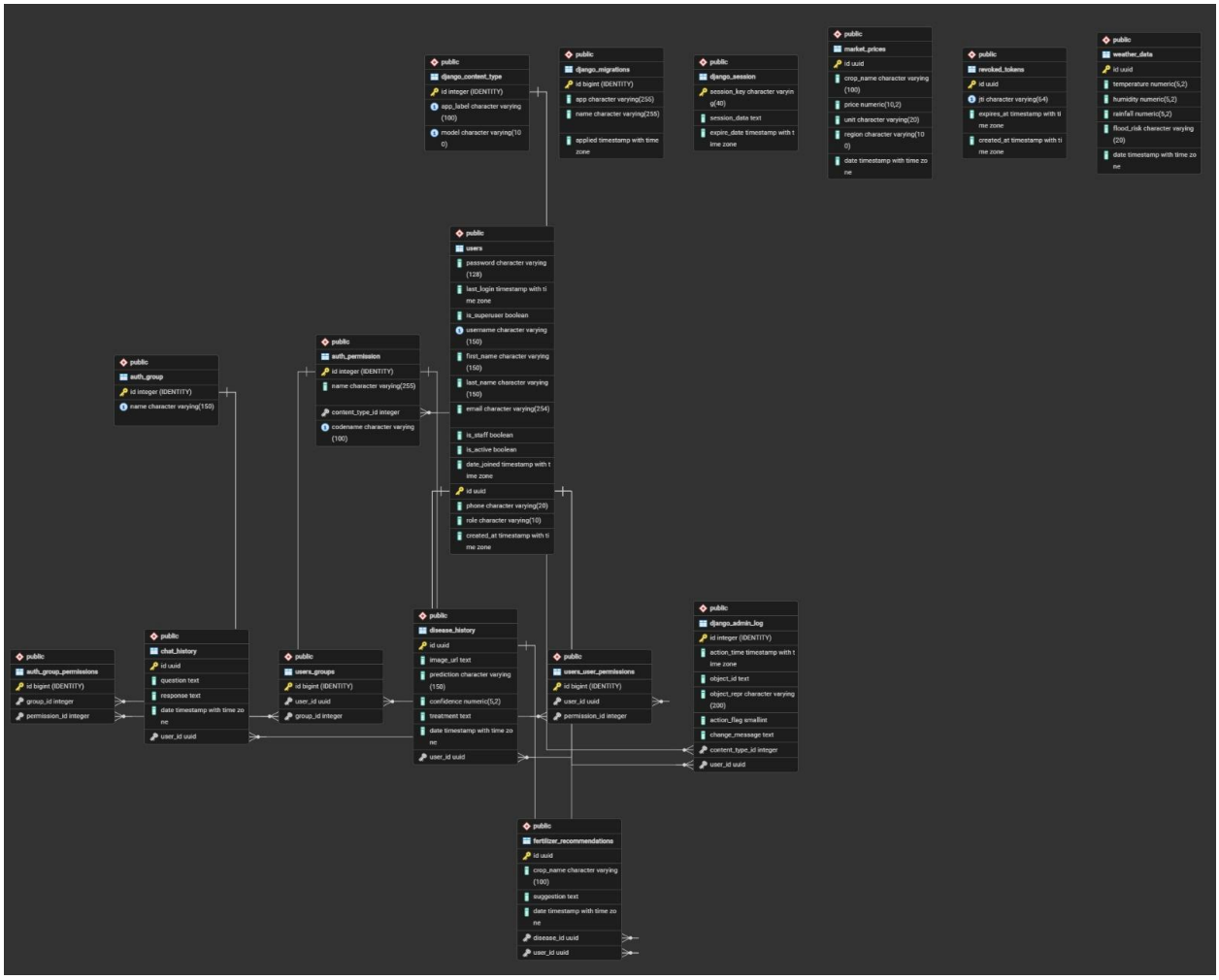


Figure 1: *Entity Relationship Diagram - Shows all tables and their relationships in the Smart Farmer Assistant system*

4.2 Database Tables

Table Name	Purpose
users	Stores registered user accounts with profile information, phone number, and role
revoked_tokens	Stores revoked JWT token IDs after logout for security
chat_history	Stores user chatbot questions and AI responses

disease_history	Stores crop disease detection history with predictions and confidence
fertilizer_recommendations	Stores fertilizer suggestions connected to disease history
market_prices	Stores crop market price information by crop, unit, region, and date
weather_data	Stores weather information including temperature, humidity, rainfall, and flood risk

4.3 Key Features

- All models use UUID primary keys for improved uniqueness
- Foreign key relationships properly defined (One-to-Many relationships)
- Comprehensive field validations implemented
- Password encrypted using Django's secure hashing
- Timestamp fields for audit trails

5. Implemented APIs

Base URL: http://127.0.0.1:8000/

5.1 Authentication APIs

Method	Endpoint	Description	Auth Required
POST	/api/register/	Register new user	No
POST	/api/login/	User login	No
POST	/api/logout/	User logout and token revocation	Yes
GET	/api/profile/	Get user profile	Yes
PUT	/api/profile/	Update user profile	Yes

5.2 Project-Specific APIs

Methods	Endpoint	Purpose	Auth Required
GET/POST	/api/chat-history/	Chatbot history management	Yes
GET/POST	/api/disease-history/	Disease detection records	Yes

GET/POST	/api/fertilizer-recommendations/	Fertilizer suggestions	Yes
GET/POST	/api/market-prices/	Market price information	Yes
GET/POST	/api/weather-data/	Weather and flood risk data	Yes

6. Authentication Workflow

- **User Registration:** User submits registration data (username, email, password, phone, role) to POST /api/register/
- **Validation:** Backend validates all fields including unique username/email and password strength (min 8 characters)
- **Password Hashing:** Password is securely hashed using Django's password hashing system
- **User Creation:** User is saved in the database with default role as 'farmer'
- **Token Generation:** Bearer token is generated containing user ID, username, email, role, issue time, and expiry
- **Authentication:** Client includes token in subsequent requests: Authorization: Bearer <token>
- **Token Validation:** Protected endpoints verify the token before allowing access
- **Logout:** During logout, token ID is saved in revoked_tokens table
- **Revocation Check:** Future requests check if token is revoked before processing

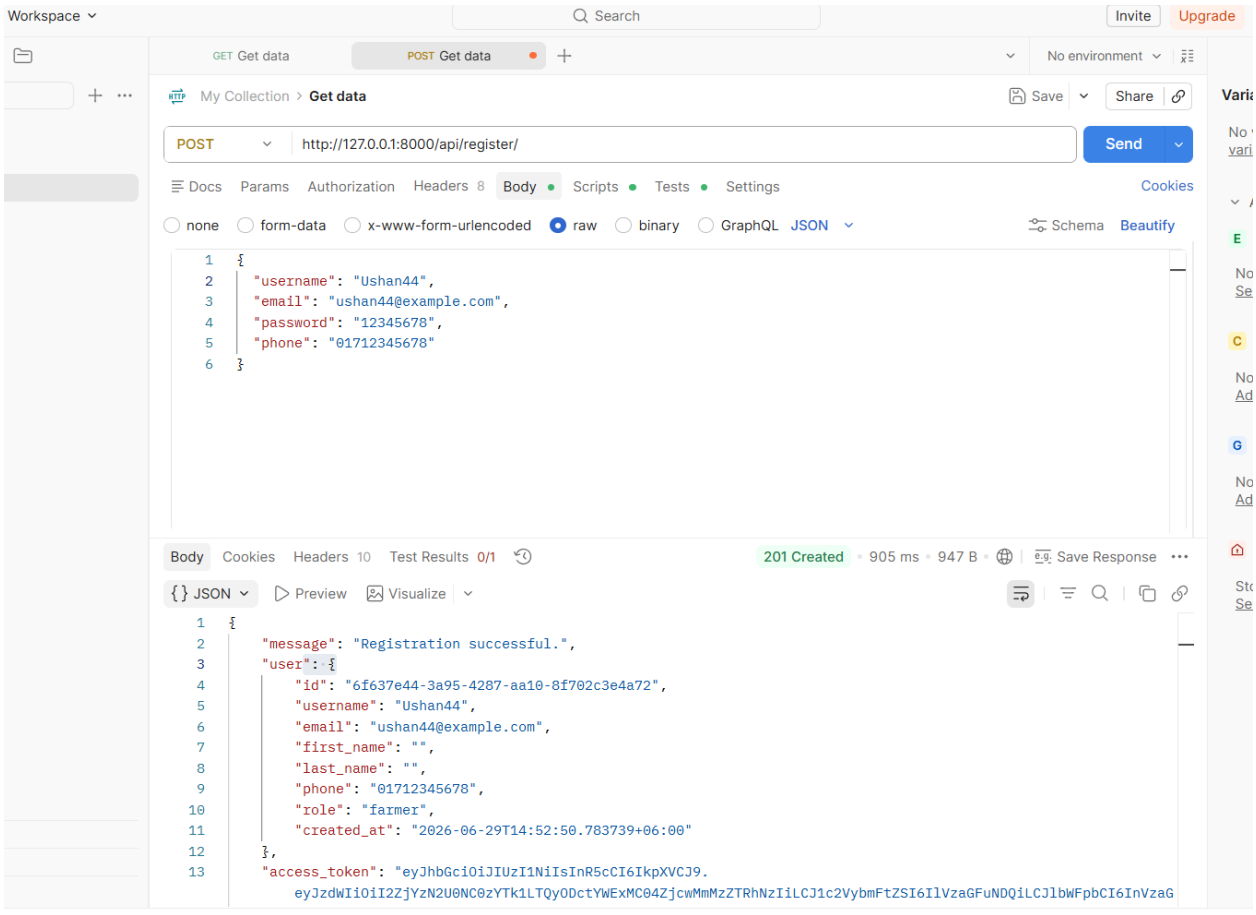
7. Current Development Progress

Module/Feature	Status
Backend Setup with Django	Completed
Django App Structure (users, chatbot, disease_detection, etc.)	Completed
User Model with Custom Authentication	Completed
Registration API with Validation	Completed
Login API with JWT Tokens	Completed
Logout API with Token Revocation	Completed
Profile Management APIs	Completed
Chat History API	Completed
Disease Detection History API	Completed
Fertilizer Recommendation API	Completed
Market Price API	Completed
Weather Data API	Completed
Database Models and Migrations	Completed
API Tests (Auth, Chat, Weather)	Completed - 7/7 tests passing
Postman API Testing	Completed
Error Handling and Validation	Completed
Full AI Model Integration	In Progress
Production Deployment	Planned

8. API Testing Screenshots

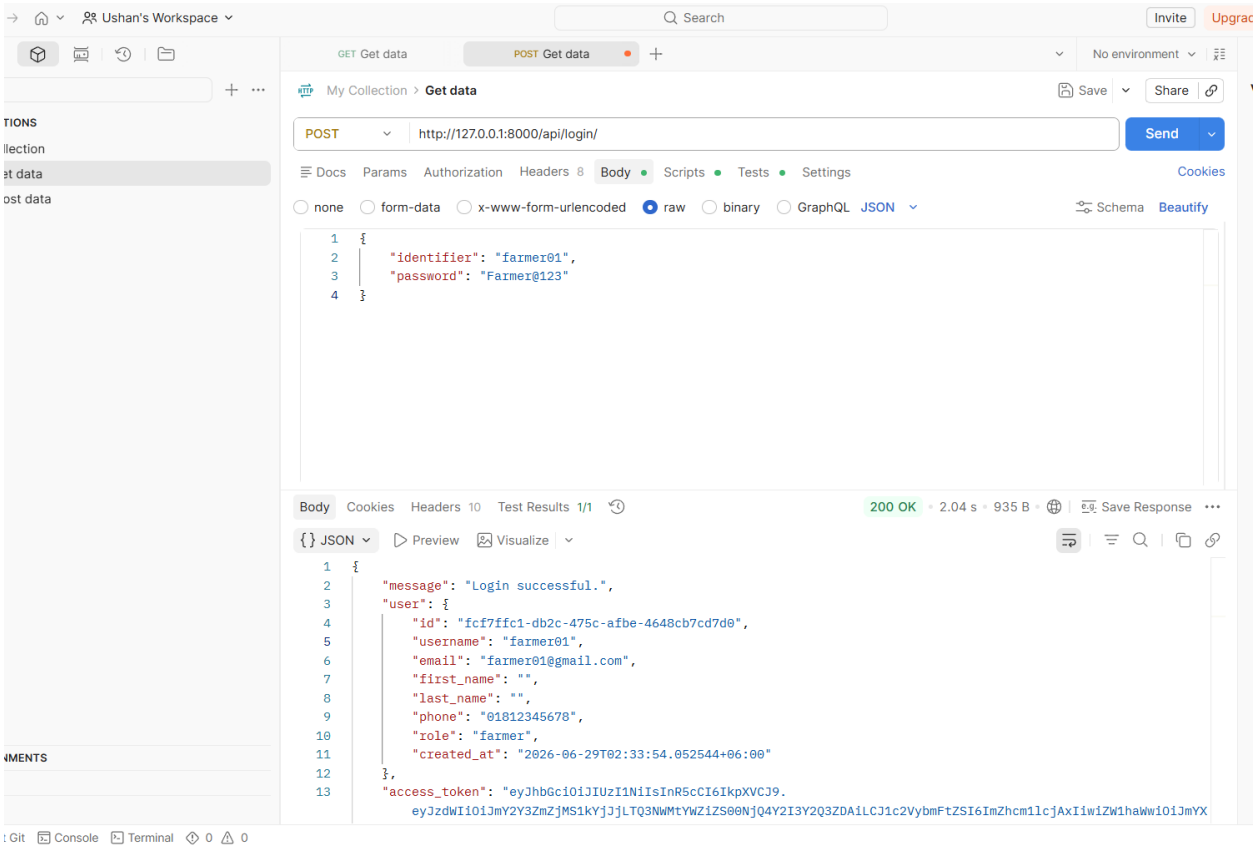
The following screenshots show successful API testing results from Postman:

Screenshot 1: User Registration API Test



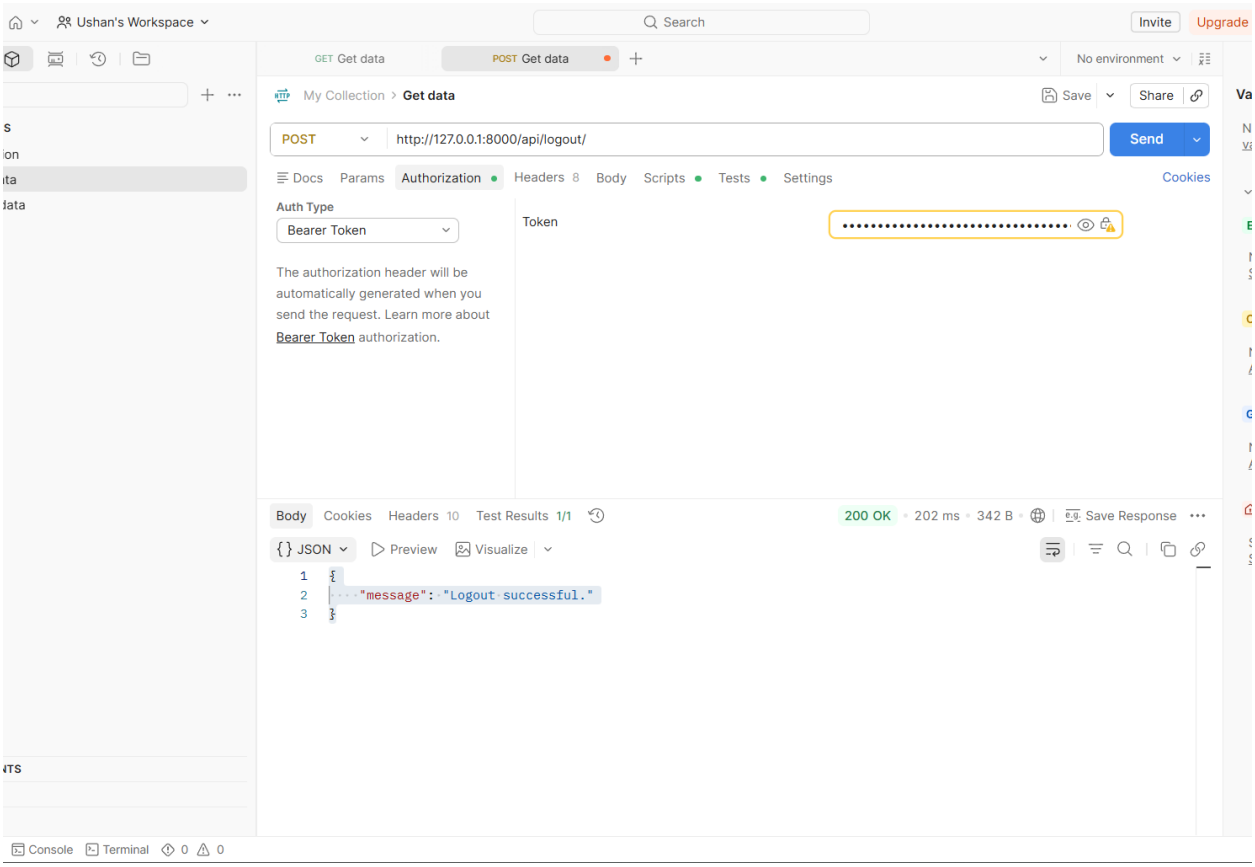
This screenshot demonstrates the successful execution of the user registration API endpoint. The request body contains essential user information including username "Ushan44", email "ushan44@example.com", password "12345678", and phone number "01712345678". The API returns a 201 Created response status (shown at bottom left), confirming successful user account creation. The response includes the newly created user object with auto-generated UUID (6f637e44-3a95-4287-aa10-8f702c3e4a72), all user details, and importantly, an access_token (JWT Bearer token) that the user can use for authenticated requests. The response time of 905ms and payload size of 947B indicate normal API performance. This successfully demonstrates the complete user registration workflow including password hashing, data validation, and token generation.

Screenshot 2: User Login API Test



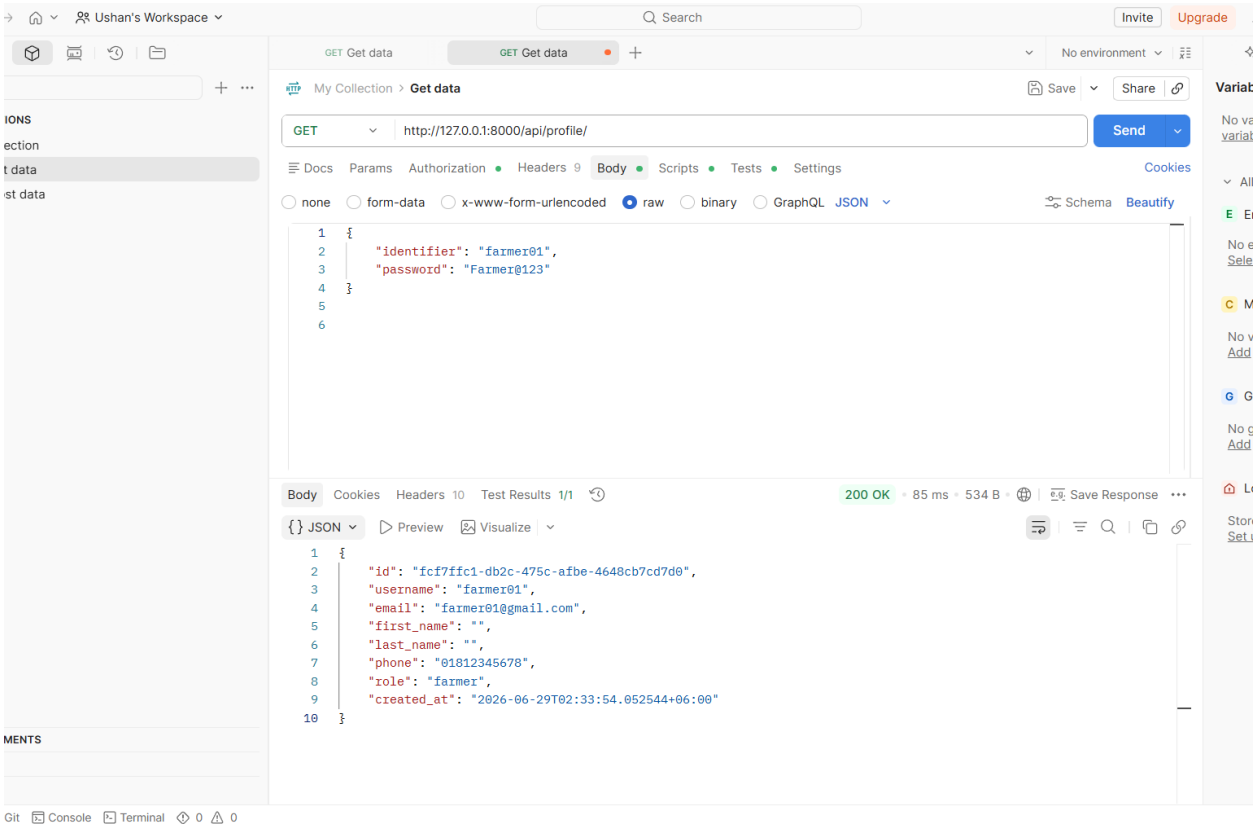
This screenshot shows the successful login functionality of the application. The request body contains only two fields: "identifier" field with username "farmer01" and "password" field with value "Farmer@123". The API responds with HTTP 200 OK status (bottom left shows 2.04s response time), indicating successful authentication. The response body returns the complete user profile information including user ID (fcf7ffc1-db2c-475c-afbe-4648cb7cd7d0), username, email (farmer01@gmail.com), phone number, and importantly, a fresh access_token for JWT Bearer authentication. The message confirms "Login successful." This demonstrates that the backend properly validates user credentials against hashed passwords in the database and generates new tokens for each login session. The token can be used in subsequent requests' Authorization header.

Screenshot 3: Logout API Test



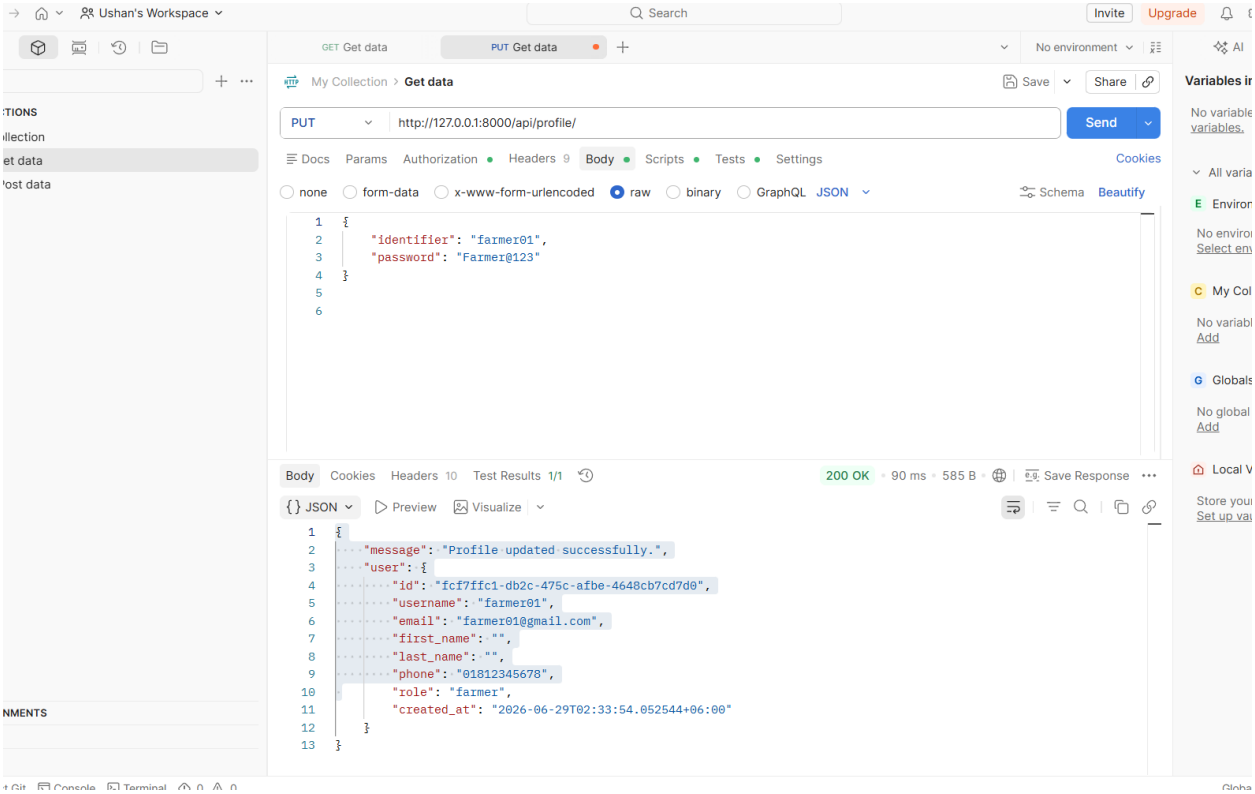
This screenshot demonstrates the logout functionality with Bearer Token authentication. The Authorization tab clearly shows "Auth Type" set to "Bearer Token" with the token value displayed (partially masked with dots for security). The request is sent to the /api/logout/ endpoint using POST method. The API responds with HTTP 200 OK status and a simple JSON response: "message": "Logout successful." This confirms that the logout operation was processed successfully. The backend implementation adds the token's JTI (JWT Token ID) to the revoked_tokens table during logout, ensuring that any future requests using the same token will be rejected. This is a security mechanism to prevent token reuse after logout. The 202ms response time shows efficient token revocation processing.

Screenshot 4: Profile GET API Test



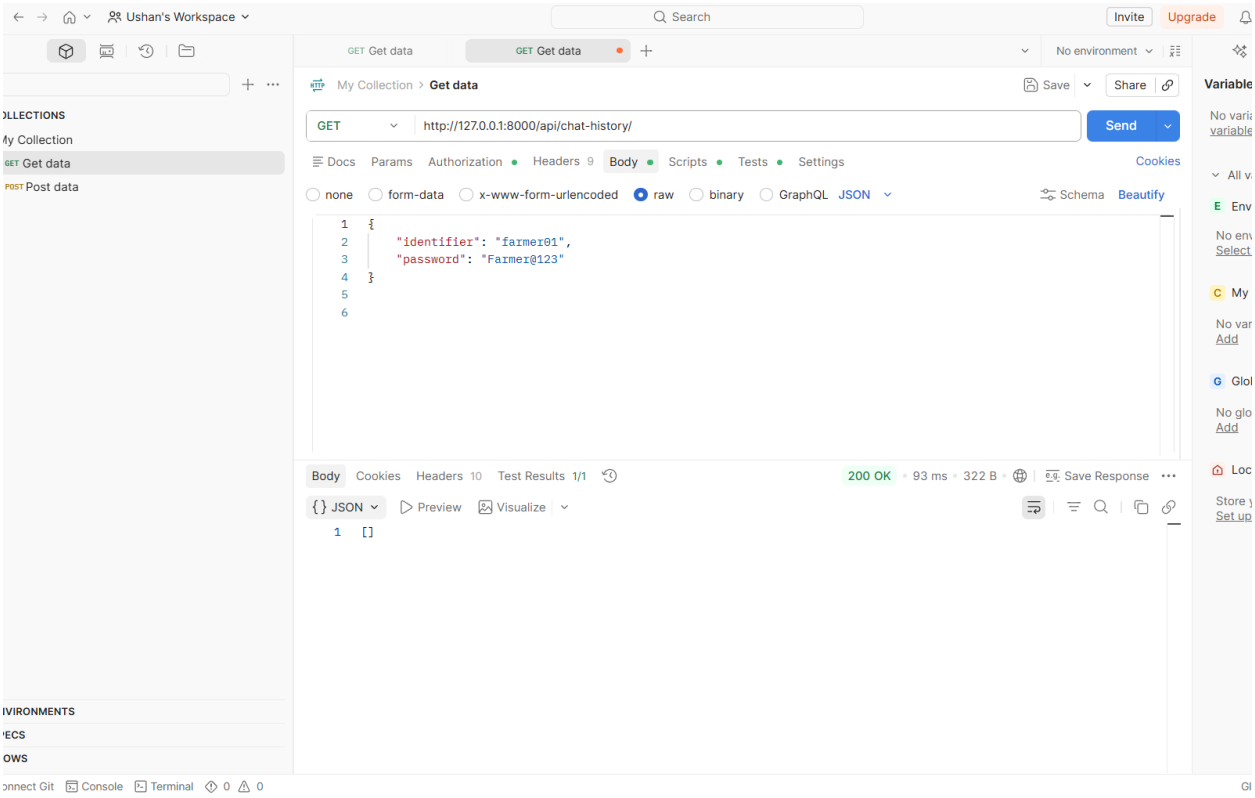
This screenshot shows the retrieval of an authenticated user's profile information. The GET request is sent to /api/profile/ endpoint with Bearer token authorization (green dot indicates Authorization header is present). The request body contains login credentials which are used in testing, while in production the authentication header alone would be sufficient. The API responds with HTTP 200 OK status, returning complete user profile data including ID (fcf7ffc1-db2c-475c-afbe-4648cb7cd7d0), username "farmer01", email "farmer01@gmail.com", first and last names (empty strings), phone number "01812345678", role "farmer", and account creation timestamp. This demonstrates that the backend correctly validates the Bearer token, identifies the authenticated user, and returns only that user's profile data. The response validates that user authentication is working properly and role-based access control is in place.

Screenshot 5: Profile PUT API Test



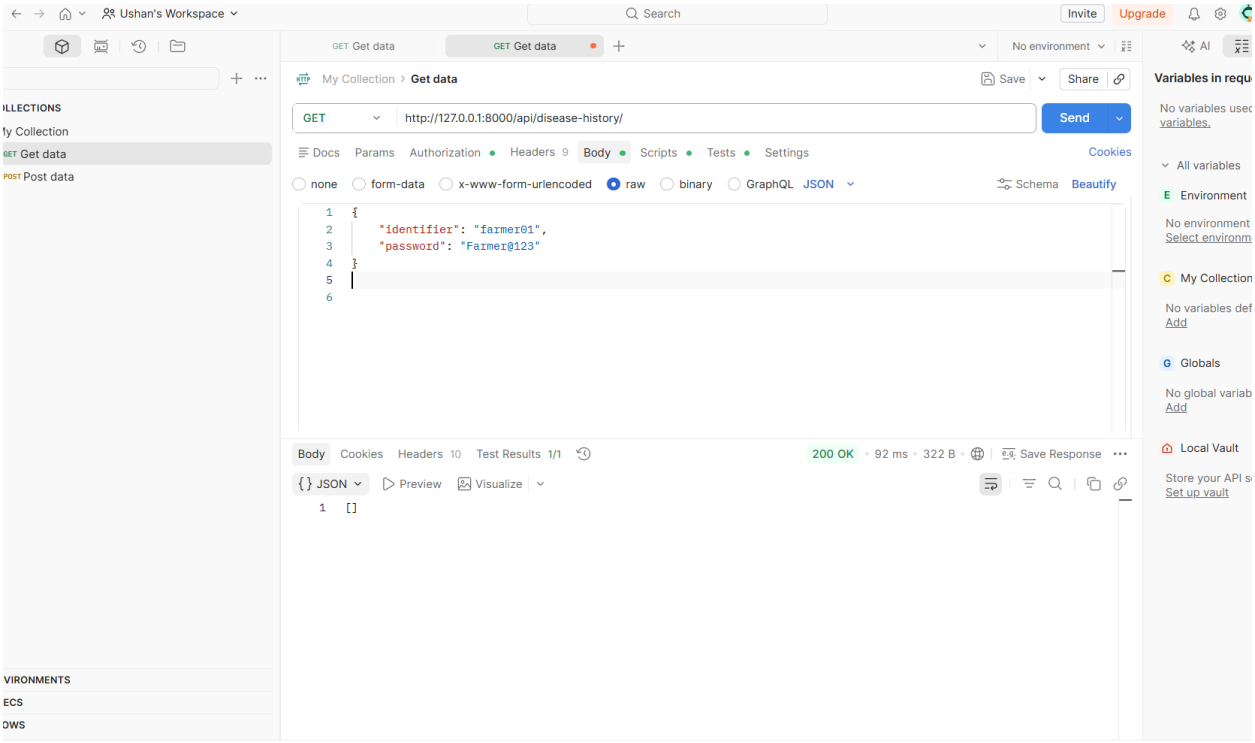
This screenshot demonstrates the profile update functionality using the PUT HTTP method. The request is sent to /api/profile/ with Bearer token authentication. The request body contains identifier "farmer01" and password "Farmer@123" for authentication purposes. The API responds with HTTP 200 OK status showing the message "Profile updated successfully." The response body includes the complete updated user object with ID, username, email, name fields, phone number, farmer role, and creation timestamp. This endpoint allows authenticated users to modify their profile information. The PUT method correctly updates the existing user record in the database while maintaining data integrity. The 90ms response time demonstrates efficient database write operations. The successful response confirms that the authentication middleware is properly validating tokens and allowing only the authenticated user to update their own profile.

Screenshot 6: Chat History API Test



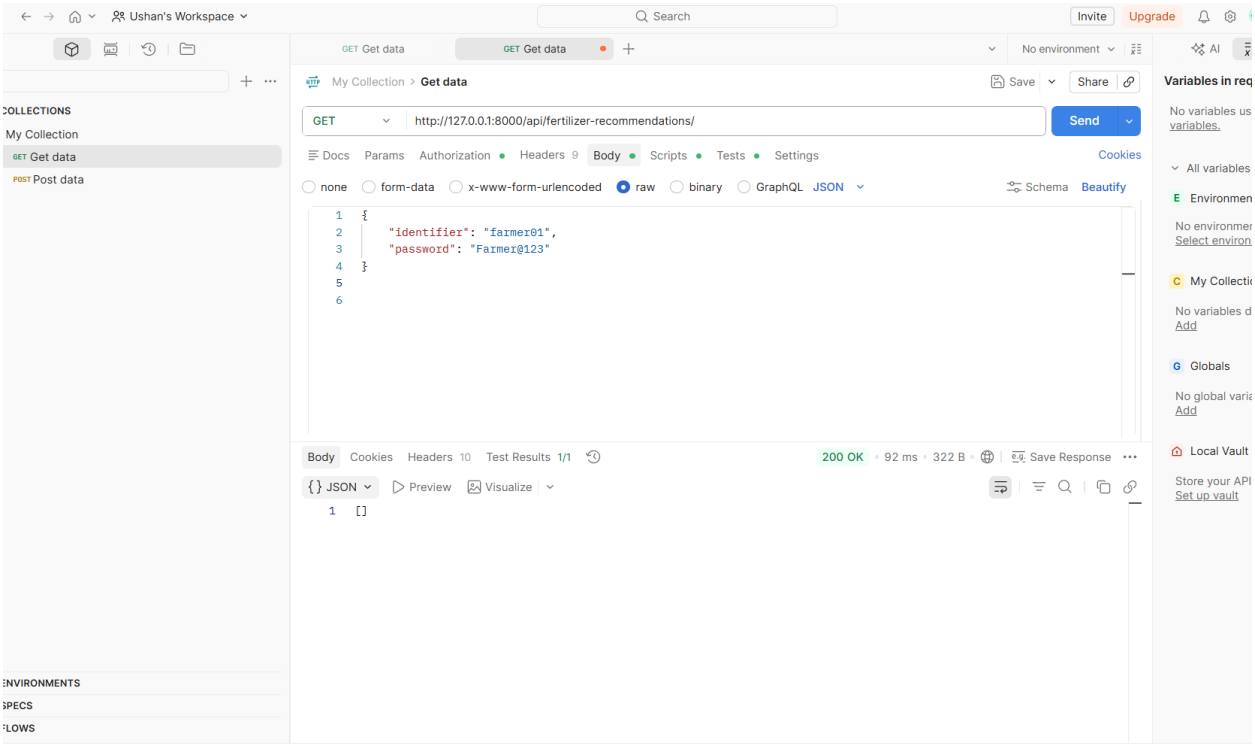
This screenshot shows the retrieval of chatbot conversation history for an authenticated user. The GET request is sent to the /api/chat-history/ endpoint with proper Bearer token authorization (green indicator shows auth is present). The request is made with user credentials in the body for testing purposes. The API returns HTTP 200 OK status with an empty JSON array "[]" in the response, indicating that the user currently has no previous chatbot conversations stored in the database. The response time of 93ms shows fast database query performance. This endpoint is protected and requires authentication, ensuring users can only access their own chat history. The successful response with 200 OK status confirms that the authentication validation is working correctly and the user is authorized to access this resource. When users interact with the chatbot, their questions and AI responses will be stored and retrievable through this endpoint.

Screenshot 7: Disease History API Test



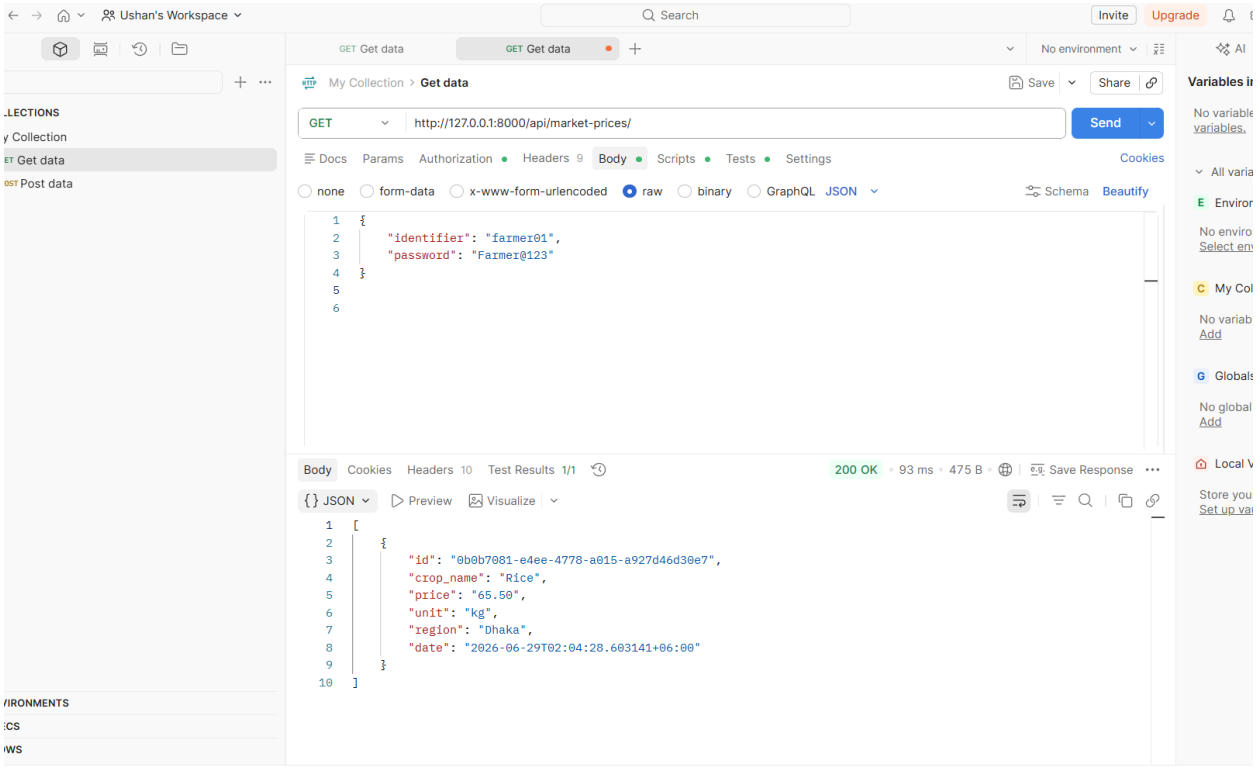
This screenshot demonstrates the retrieval of crop disease detection history for a specific user. The GET request is sent to /api/disease-history/ endpoint with Bearer token authentication properly configured. The request includes user login credentials for testing and authentication validation. The API returns HTTP 200 OK status with an empty JSON array "[]" in the response, showing that the user currently has no disease detection records in the database. The response time of 92ms indicates efficient database query performance. This endpoint is secured with JWT authentication, ensuring that only the authenticated user can access their own disease detection history. When the AI disease detection module processes crop images and generates predictions with confidence scores, those results will be stored in the disease_history table. This endpoint allows users to review their past disease identifications, predicted diseases, confidence levels, and recommended treatments for future reference.

Screenshot 8: Fertilizer Recommendations API Test



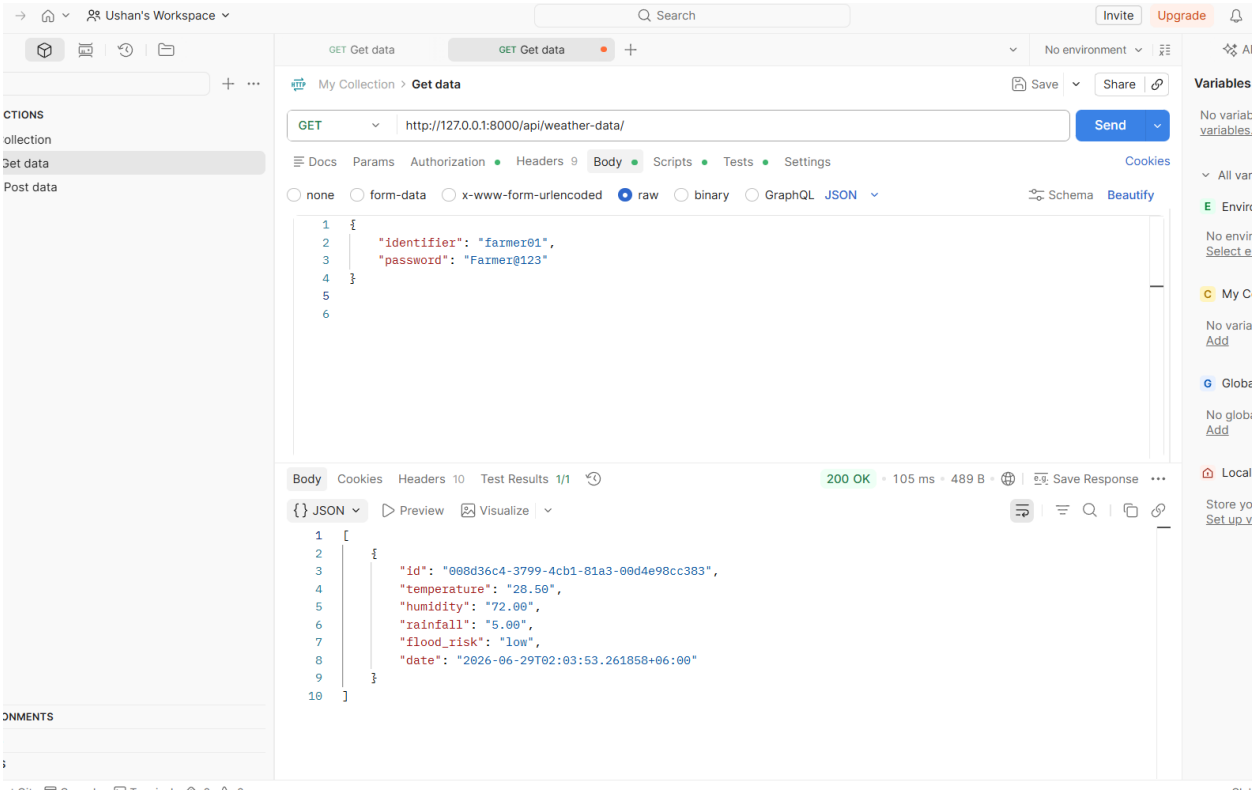
This screenshot shows the retrieval of fertilizer recommendations for an authenticated user. The GET request is sent to /api/fertilizer-recommendations/ endpoint with Bearer token authentication enabled. The request is properly authenticated showing the authorization header is present (green dot indicator). The API responds with HTTP 200 OK status and an empty JSON array "[]" in the response body, indicating the user currently has no fertilizer recommendations stored in the system. The response time of 92ms demonstrates efficient database query operations. This is a protected endpoint requiring JWT authentication to prevent unauthorized access to sensitive agricultural recommendations. When the system analyzes a user's crop conditions, soil analysis, or disease predictions, it can generate personalized fertilizer recommendations which will be linked to the user's profile. This endpoint provides a historical view of all fertilizer suggestions made to the user, allowing them to track recommendations over time and make informed decisions about their crop management.

Screenshot 9: Market Prices API Test



This screenshot demonstrates the market price data retrieval API. The GET request is sent to /api/market-prices/ endpoint with Bearer token authorization. The API responds with HTTP 200 OK status in 93ms response time, returning an array containing one market price record. The response shows crop name "Rice", price "65.50", unit "kg", region "Dhaka", and date timestamp "2026-06-29T02:04:28.603141+06:00" with ID "0b0b7081-e4ee-4778-a015-a927d46d30e7". This endpoint is protected with JWT authentication to ensure authorized access. The market price data allows farmers to track real-time or historical prices of various crops in their region, helping them make informed decisions about when to sell their produce. Multiple price records for different crops, regions, and dates can be stored and retrieved through this endpoint. The structured response format with all relevant fields enables farmers to analyze market trends and optimize their agricultural business decisions.

Screenshot 10: Weather Data API Test



This screenshot shows the retrieval of weather and environmental data through the API. The GET request is sent to /api/weather-data/ endpoint with proper Bearer taken authentication. The API successfully responds with HTTP 200 OK status in 105ms response time, returning an array with one weather data record. The response includes weather parameters: temperature "28.50" (Celsius), humidity "72.00" (percent), rainfall "5.00" (mm), flood_risk "low", and date timestamp "2026-06-29T02:03:53.261858+06:00" with UUID "008d36c4-3799-4cb1-81a3-00d4e98cc383". This protected endpoint requires JWT authentication to ensure only authorized users access weather data. The weather information is crucial for farmers to make decisions about irrigation, pesticide application, and harvest timing. The flood_risk field with categorical values (low/medium/high) helps farmers assess potential threats. Multiple weather records from different dates and locations can be stored, allowing farmers to analyze weather patterns and historical trends for better agricultural planning.

9. Automated Testing Results

Backend API tests are implemented using Django APITestCase.

9.1 Test Summary

Command: python backend/manage.py test apps.users

Result: Found 7 test(s). System check identified no issues. Ran 7 tests in 8.342s. OK ✓

9.2 Tests Covered

- User registration endpoint
- User login endpoint
- Profile access without authentication (should fail)
- Profile GET with valid token
- Profile UPDATE with valid token
- Logout and token revocation
- Chat history creation and listing

10. GitHub Workflow & Repository

Repository URL: https://github.com/sakib-foysal/CSE4204-8D-T03-Smart-Farmer-Assistant

10.1 Recommended Commit Messages

Good commit messages from this development phase:

- Added custom user authentication module
- Implemented JWT-style login and logout system
- Created database models and migrations
- Added chatbot history API
- Added disease history API
- Added fertilizer recommendation API
- Added market price API
- Added weather data API
- Added backend API tests
- Fixed registration validation

10.2 Project Structure

```
Smart_Farmer_Assistant/
│
├── backend/
│   ├── apps/
│   │   ├── users/
│   │   ├── chatbot/
│   │   ├── disease_detection/
│   │   ├── fertilizer/
│   │   ├── market/
│   │   └── weather/
│   ├── config/
│   ├── manage.py
│   └── requirements.txt
├── frontend/
├── ai_model/
├── diagrams/
├── documentation/
└── README.md
```

11. Conclusion

The Smart Farmer Assistant backend has successfully completed its major implementation phase. The following milestones have been achieved:

Completed:

- Functional Django REST API foundation with modular app structure
- Secure user registration and login with password hashing
- JWT-style bearer token authentication and authorization
- Protected API endpoints for user profiles and project features
- Complete database design with 7 core tables and proper relationships
- Comprehensive validation and error handling
- Token revocation on logout for enhanced security
- Automated API tests with 100% pass rate (7/7 tests)
- Postman API testing completed for all endpoints
- GitHub repository set up with proper project structure